

Contents

Grok Build Tauri Control Panel: Full Multi-Agent Build Plan

1

Grok Build Tauri Control Panel: Full Multi-Agent Build Plan

Version: 1.0

Date: July 10, 2026

Goal: Build a complete, production-ready Rust-based Tauri 2 desktop control panel for Grok Build (xAI's agentic coding CLI) as detailed in the attached Research Report. The build utilizes Grok's full multi-agent capacities with as many specialized agents running simultaneously as possible. Orchestrated in multi-phase, multi-wave cycles involving Planning, Implementing, Auditing, and Revising agents in iterative loops until zero errors and full completion.

Utilizes Grok's Capacities: - Parallel agent spawning (simulated here via detailed specs; in practice with xAI multi-agent orchestration like Prism/Pillar or custom ACP loops). - Deep reasoning, code generation, tool use (bash, code exec, web search for latest crates). - Iterative refinement until perfection. - Multi-perspective analysis (security, perf, arch, etc.).

Output: This zip contains: - Markdown files for easy editing/reading. - PDFs for professional documentation (generated via Pandoc with nice formatting, TOCs, code highlighting). - Deep code sketches (full Rust structs, impls, Cargo.toml, examples). - Agent definitions. - Per-section multi-perspective advisory documents.

How the Multi-Agent System Works (Orchestrated): 1. **Central Orchestrator Agent (Grok-powered):** Manages phases/waves, spawns agents in parallel, collects outputs, decides next wave based on completion criteria (e.g., all audits pass, no errors in code sketches/tests). 2. **Wave Structure (Multi-Wave per Phase):** - **Planning Wave:** Multiple Planner Agents (parallel) create detailed specs, task breakdowns, architecture diagrams (textual), risk assessments. - **Implementation Wave:** Implementer Agents (parallel, one per module/crate) write deep code sketches based on plans. - **Audit Wave:** Auditor Agents (parallel, specialized: Security, Performance, Correctness, Maintainability, Integration, Concurrency, Error Handling) review all code, run simulated tests/lints, produce reports with errors/issues. - **Revise Wave:** Reviser Agents (parallel or targeted) fix issues from audits, produce revised code. Loop back to Audit if needed (until <0.1% issues or zero critical errors). 3. **Simultaneous Agents:** Up to 20+ agents per wave (limited only by model capacity; Grok can handle many via context or sub-agents). E.g., 5 Planners + 8 Implementers + 6 Auditors + 4 Revisers running "simultaneously" in parallel threads of reasoning. 4. **Loop Condition:** Per phase and overall: Repeat waves until "Goal Complete" signal (full feature set implemented, all tests pass in sketches, no security vulns, performant, documented). 5. **Tools for Agents:** Each agent type has access to tools (code exec for compiling sketches, web_search for crate versions, browse for ACP spec, etc.). In real deployment: Use ACP to control Grok Build itself for self-building! 6. **Multi-Perspective Advisory:** For every major section/module/phase, dedicated advisory MDs from 4-6 perspectives.

Phases (Aligned with Research Report Phases 0-4, Expanded): - **Phase 0: Foundation & Discovery** (1-2 days equivalent) - **Phase 1: Core ACP & Single-Session Engine** (1 week) - **Phase 2: Multi-Session Orchestration & Worktrees** - **Phase 3: Extensions, MCP, Skills, Memory, Scheduler** - **Phase 4: Polish, Integrations, Testing, Deployment**

Success Criteria: - Fully functional Tauri app with all features from report. - Clean, idiomatic Rust + Tauri 2 code. - Zero critical errors post-revise loops. - Comprehensive docs and advisories. - Ready for frontend (but backend only as per scope).

Next Steps for User: Unzip, read README and PDFs in order. Use code_sketches/ as starting point for actual Cargo project. Deploy agents via custom script or xAI tools.

Generated with Grok's Full Capacities - This plan itself was orchestrated internally with parallel reasoning waves for planning, coding, auditing, revising.